PYTHON PROGRAMMING CASE STUDIES

Algorithms, Pseudocode, Programs, Test Cases and Outputs

CASE STUDY 12: ATTENDANCE RECOVERY COMPUTATION

1. PROBLEM STATEMENT

Calculate a student's attendance percentage and find the minimum number of consecutive classes needed to reach the required attendance.

2. OBJECTIVES

• Calculate current attendance.
• Check eligibility.
• Find the minimum additional classes required.

3. ASSUMPTIONS

Required attendance is 75%. The student attends every future class.

4. ALGORITHM

1. Read total classes and attended classes.
2. Calculate attendance percentage.
3. If attendance is at least 75%, display eligible.
4. Otherwise, repeatedly increase attended and total classes by one.
5. Stop when attendance reaches 75%.
6. Display the number of classes required.

5. FLOWCHART (TEXT REPRESENTATION)

```
START
  ↓
Input total classes and attended classes
  ↓
Calculate attendance percentage
  ↓
Is attendance at least 75%?
  ↓
YES → Display eligible
  ↓
NO → Attend one more class; recalculate
  ↓
STOP
```

6. PSEUDOCODE

```
START
INPUT total, attended
IF total <= 0 OR attended < 0 OR attended > total
    PRINT "Invalid input"
ELSE
    classes = 0
    WHILE attended * 100 < 75 * total
        attended = attended + 1
        total = total + 1
        classes = classes + 1
    PRINT classes
STOP
```

7. PYTHON PROGRAM

```
total = int(input("Enter total classes: "))
attended = int(input("Enter attended classes: "))

if total <= 0 or attended < 0 or attended > total:
    print("Invalid input")
else:
    original_total = total
    original_attended = attended
    classes = 0
    while attended * 100 < 75 * total:
        attended += 1
        total += 1
        classes += 1

    print("Current attendance:",
          round(original_attended * 100 / original_total, 2), "%")
    print("Classes required:", classes)
    print("Final attendance:", round(attended * 100 / total, 2), "%")
```

8. TEST CASE

Total classes = 20, attended = 12.

9. OUTPUT

```
Current attendance: 60.0 %
Classes required: 12
Final attendance: 75.0 %
```

10. CONCLUSION

The program calculates attendance and determines the minimum classes required to achieve 75% attendance.

CASE STUDY 13: ATM TRANSACTION SEQUENCE SIMULATION

1. PROBLEM STATEMENT

Simulate deposits and withdrawals, reject invalid transactions, and display the final account balance.

2. OBJECTIVES

• Use lists, loops, conditional statements, and arithmetic operators.

3. ASSUMPTIONS

Opening balance is ■5,000; each withdrawal cannot exceed the available balance.

4. ALGORITHM

1. Initialize the balance.
2. Read the number of transactions.
3. For each transaction, read its type and amount.
4. Accept valid deposits.
5. Accept withdrawals only when the amount is positive and within the balance.
6. Reject invalid transactions.
7. Display the final balance.

5. FLOWCHART (TEXT REPRESENTATION)

```
START
  ↓
Initialize balance and read transaction count
  ↓
Read transaction type and amount
  ↓
Deposit or withdrawal?
  ↓
Deposit: add amount. Withdrawal: check balance. Invalid: reject.
  ↓
More transactions? Repeat until complete.
  ↓
Display balance → STOP
```

6. PSEUDOCODE

```
START
balance = 5000
INPUT n
REPEAT n times
    INPUT type, amount
    IF amount <= 0
        PRINT "Invalid amount"
    ELSE IF type is deposit
        balance = balance + amount
    ELSE IF type is withdrawal AND amount <= balance
        balance = balance - amount
    ELSE
        PRINT "Transaction rejected"
PRINT balance
STOP
```

7. PYTHON PROGRAM

```
balance = 5000
transactions = []
n = int(input("Number of transactions: "))

for i in range(n):
    kind = input("Enter deposit or withdraw: ").lower()
    amount = float(input("Enter amount: "))

    if amount <= 0:
        print("Invalid amount")
        transactions.append("Rejected")
    elif kind == "deposit":
        balance += amount
        transactions.append("Deposit accepted")
    elif kind == "withdraw" and amount <= balance:
        balance -= amount
        transactions.append("Withdrawal accepted")
    elif kind == "withdraw":
        print("Insufficient balance")
        transactions.append("Rejected")
    else:
        print("Invalid transaction type")
        transactions.append("Rejected")

print("Transaction summary:", transactions)
print("Final balance: Rs.", balance)
```

8. TEST CASE

Opening balance ■5,000; deposit ■1,000; withdrawal ■2,000.

9. OUTPUT

```
Transaction summary: ['Deposit accepted', 'Withdrawal accepted']
Final balance: Rs. 4000.0
```

10. CONCLUSION

The program processes ATM transactions, validates amounts, and maintains the correct balance.

CASE STUDY 14: WAREHOUSE INVENTORY RECONCILIATION

1. PROBLEM STATEMENT

Calculate closing stock, compare it with recorded stock, detect discrepancies, and determine replenishment requirements.

2. OBJECTIVES

• Apply arithmetic operations, lists, loops, and conditional statements.

3. ASSUMPTIONS

Replenishment is required when stock is below the minimum level of 10 units.

4. ALGORITHM

1. Read the number of products.
2. For each product, input opening stock, received stock, dispatched stock, recorded closing stock, and minimum stock.
3. Calculate expected closing stock.
4. Compare expected and recorded stock.
5. Calculate replenishment quantity.
6. Display the results.

5. FLOWCHART (TEXT REPRESENTATION)

```
START
  ↓
Input opening, received, dispatched and recorded stock
  ↓
Expected = opening + received − dispatched
  ↓
Compare expected and recorded stock; calculate shortage
  ↓
Display report → STOP
```

6. PSEUDOCODE

```
START
INPUT number of products
FOR each product
    INPUT opening, received, dispatched, recorded, minimum
    expected = opening + received - dispatched
    discrepancy = expected - recorded
    replenishment = MAX(0, minimum - expected)
    DISPLAY expected, discrepancy, replenishment
STOP
```

7. PYTHON PROGRAM

```
n = int(input("Number of products: "))

for i in range(n):
    name = input("Product name: ")
    opening = int(input("Opening stock: "))
    received = int(input("Received stock: "))
    dispatched = int(input("Dispatched stock: "))
    recorded = int(input("Recorded closing stock: "))
    minimum = int(input("Minimum stock level: "))

    expected = opening + received - dispatched
    discrepancy = expected - recorded
    replenishment = max(0, minimum - expected)

    print("\nProduct:", name)
    print("Expected closing stock:", expected)
    print("Discrepancy:", discrepancy)
    print("Replenishment required:", replenishment)
```

8. TEST CASE

Opening = 20, received = 15, dispatched = 10, recorded = 24, minimum = 10.

9. OUTPUT

```
Expected closing stock: 25
Discrepancy: 1
Replenishment required: 0
```

10. CONCLUSION

The program identifies stock discrepancies and calculates replenishment needs.

CASE STUDY 17: STUDENT PERFORMANCE TREND ANALYSIS

1. PROBLEM STATEMENT

Compare student scores across examinations and identify improvements, declines, and weak subjects.

2. OBJECTIVES

• Calculate score changes, averages, and performance categories.

3. ASSUMPTIONS

Scores are out of 100. An average of 75 or more is classified as Good; otherwise, Needs Improvement.

4. ALGORITHM

1. Input the number of subjects.
2. Enter each subject's previous and current marks.
3. Calculate the difference and average.
4. Identify improvement or decline.
5. Display the performance category.

5. FLOWCHART (TEXT REPRESENTATION)

```
START
  ↓
Input subject and previous/current marks
  ↓
Calculate difference and average
  ↓
Difference positive, negative, or zero?
  ↓
Display improvement, decline, or no change; classify average
  ↓
STOP
```

6. PSEUDOCODE

```
START
INPUT number of subjects
FOR each subject
    INPUT previous_marks, current_marks
    difference = current_marks - previous_marks
    IF difference > 0
        PRINT "Improved"
    ELSE IF difference < 0
        PRINT "Declined"
    ELSE
        PRINT "No change"
    average = (previous_marks + current_marks) / 2
    IF average >= 75
        PRINT "Good"
    ELSE
        PRINT "Needs Improvement"
STOP
```

7. PYTHON PROGRAM

```
n = int(input("Number of subjects: "))
total_average = 0

for i in range(n):
    subject = input("Subject name: ")
    previous = float(input("Previous marks: "))
    current = float(input("Current marks: "))
    difference = current - previous
    average = (previous + current) / 2
    total_average += average

    print("Subject:", subject)
    print("Change:", difference)
    if difference > 0:
        print("Trend: Improved")
    elif difference < 0:
        print("Trend: Declined")
    else:
        print("Trend: No change")

    print("Average:", average)
    if average >= 75:
        print("Performance: Good")
    else:
        print("Performance: Needs Improvement")

if n > 0:
    print("Overall average:", round(total_average / n, 2))
```

8. TEST CASE

Mathematics: previous marks = 60, current marks = 80.

9. OUTPUT

```
Subject: Mathematics
Change: 20.0
Trend: Improved
Average: 70.0
Performance: Needs Improvement
```

10. CONCLUSION

The program compares examination scores and reports student performance trends.

CASE STUDY 26: NESTED LOOP EXAMINATION SEATING GENERATOR

1. PROBLEM STATEMENT

Generate a classroom seating arrangement using rows and columns while keeping reserved seats empty.

2. OBJECTIVES

• Use nested loops, lists, and conditional statements to generate seating.

3. ASSUMPTIONS

There are 3 rows and 4 columns. Seats R1C2 and R2C3 are reserved. Students are assigned in order.

4. ALGORITHM

1. Create a list of student roll numbers.
2. Define the classroom rows and columns.
3. Traverse each seat using nested loops.
4. Keep reserved seats empty.
5. Assign the next student to each available seat.
6. Display the seating arrangement.

5. FLOWCHART (TEXT REPRESENTATION)

```
START
  ↓
Input student roll numbers and reserved seats
  ↓
Repeat for each row and column
  ↓
Is the seat reserved?
  ↓
Yes: mark reserved. No: assign next student.
  ↓
Display seating → STOP
```

6. PSEUDOCODE

```
START
INPUT student list
reserved = [(0,1), (1,2)]
index = 0
FOR each row
    FOR each column
        IF position is reserved
            DISPLAY "RESERVED"
        ELSE IF students remain
            DISPLAY next student
            index = index + 1
        ELSE
            DISPLAY "EMPTY"
STOP
```

7. PYTHON PROGRAM

```
students = input("Enter roll numbers separated by spaces: ").split()
rows = 3
cols = 4
reserved = [(0, 1), (1, 2)]
index = 0

for r in range(rows):
    for c in range(cols):
        if (r, c) in reserved:
            seat = "RESERVED"
        elif index < len(students):
            seat = students[index]
            index += 1
        else:
            seat = "EMPTY"
        print(seat, end="\t")
    print()
```

8. TEST CASE

Roll numbers: 101 102 103 104 105 106 107 108 109 110.

9. OUTPUT

```
101\tRESERVED\t102\t103
104\t105\tRESERVED\t106
107\t108\t109\t110
```

10. CONCLUSION

Nested loops help generate classroom seating while respecting reserved positions.

CASE STUDY 27: ADVANCED NUMERIC PATTERN GENERATOR

1. PROBLEM STATEMENT

Generate a hollow diamond pattern using a given odd number of rows.

2. OBJECTIVES

• Practise nested loops, conditions, and pattern generation.

3. ASSUMPTIONS

The input must be an odd integer greater than or equal to 3.

4. ALGORITHM

1. Read the number of rows.
2. Validate the input.
3. Calculate the middle row.
4. Use nested loops to print leading spaces and stars.
5. Print only the boundary stars to create a hollow diamond.

5. FLOWCHART (TEXT REPRESENTATION)

```
START
  ↓
Input number of rows
  ↓
Is rows odd and at least 3?
  ↓
No: display error. Yes: generate hollow diamond using loops.
  ↓
STOP
```

6. PSEUDOCODE

```
START
INPUT n
IF n < 3 OR n is even
    PRINT "Invalid dimensions"
ELSE
    FOR each row i from 0 to n-1
        Calculate distance from middle
        Print leading spaces
        Print boundary stars and inner spaces
        Move to next line
STOP
```

7. PYTHON PROGRAM

```
n = int(input("Enter odd number of rows: "))

if n < 3 or n % 2 == 0:
    print("Invalid dimensions")
else:
    mid = n // 2
    for i in range(n):
        level = i if i <= mid else n - 1 - i
        spaces = mid - level
        print(" " * spaces, end="")
        if level == 0:
            print("*")
        else:
            print("*" + " " * (2 * level - 1) + "*")
```

8. TEST CASE

n = 5.

9. OUTPUT

```
  *
 * *
*   *
 * *
  *
```

10. CONCLUSION

The program uses loops and conditions to generate a hollow diamond of exactly n lines.

CASE STUDY 28: REPEATED TRANSACTION DETECTION

1. PROBLEM STATEMENT

Identify duplicate transaction IDs and transactions exceeding a specified amount.

2. OBJECTIVES

• Use lists, loops, conditions, and duplicate detection.

3. ASSUMPTIONS

A transaction is suspicious if its ID repeats or its amount exceeds ■10,000.

4. ALGORITHM

1. Input transaction IDs and amounts.
2. Store IDs already encountered.
3. Check whether each ID has appeared before.
4. Check whether its amount exceeds ■10,000.
5. Add suspicious transactions to a list.
6. Display the suspicious entries.

5. FLOWCHART (TEXT REPRESENTATION)

```
START
  ↓
Input transaction IDs and amounts
  ↓
Check duplicate ID and amount threshold
  ↓
Suspicious?
  ↓
Yes: add to suspicious list. No: continue.
  ↓
Display results → STOP
```

6. PSEUDOCODE

```
START
INPUT transactions
seen = empty list
suspicious = empty list
FOR each transaction
    IF ID is already in seen OR amount > 10000
        ADD transaction to suspicious
    ADD ID to seen
DISPLAY suspicious
STOP
```

7. PYTHON PROGRAM

```
transactions = [
    ("T101", 2000),
    ("T102", 15000),
    ("T101", 3000),
    ("T103", 500)
]
seen = []
suspicious = []

for tid, amount in transactions:
    if tid in seen or amount > 10000:
        suspicious.append((tid, amount))
    seen.append(tid)

print("Suspicious transactions:")
for item in suspicious:
    print(item)
```

8. TEST CASE

Transactions are the four entries in the program.

9. OUTPUT

```
Suspicious transactions:
('T102', 15000)
('T101', 3000)
```

10. CONCLUSION

The program detects repeated IDs and unusually large transactions.

CASE STUDY 35: TEXT COMPRESSION USING CHARACTER RUNS

1. PROBLEM STATEMENT

Compress consecutive repeated characters and compare the original and compressed string lengths.

2. OBJECTIVES

• Practise strings, loops, and character comparison.

3. ASSUMPTIONS

Each repeated run is encoded as the character followed by its count. If compression is not shorter, retain the original string.

4. ALGORITHM

1. Input a string.
2. Count consecutive occurrences of each character.
3. Append each character and its count to the encoded string.
4. Compare encoded and original lengths.
5. Display the shorter representation.

5. FLOWCHART (TEXT REPRESENTATION)

```
START
  ↓
Input string
  ↓
Count consecutive identical characters
  ↓
Build encoded string
  ↓
Is encoded string shorter?
  ↓
Yes: use encoded string. No: retain original.
  ↓
Display result → STOP
```

6. PSEUDOCODE

```
START
INPUT string
encoded = empty string
i = 0
WHILE i < length of string
    count consecutive identical characters
    APPEND character and count to encoded
    MOVE i to next different character
IF length(encoded) < length(string)
    DISPLAY encoded
ELSE
    DISPLAY original string
STOP
```

7. PYTHON PROGRAM

```
s = input("Enter a string: ")
encoded = ""
i = 0

while i < len(s):
    count = 1
    while i + count < len(s) and s[i] == s[i + count]:
        count += 1
    encoded += s[i] + str(count)
    i += count

print("Original string:", s)
print("Encoded string:", encoded)
if len(encoded) < len(s):
    print("Final result:", encoded)
else:
    print("Compression not beneficial")
    print("Final result:", s)
print("Original length:", len(s))
print("Encoded length:", len(encoded))
```

8. TEST CASE

Input aaabbcccc.

9. OUTPUT

```
Original string: aaabbcccc
Encoded string: a3b2c4
Final result: a3b2c4
Original length: 9
Encoded length: 6
```

10. CONCLUSION

Run-length encoding compresses consecutive repeated characters and identifies when compression is beneficial.

CASE STUDY 42: STUDENT MARKS MATRIX ANALYSIS

1. PROBLEM STATEMENT

Analyse student marks stored in a matrix and calculate student totals, subject averages, and the top performer.

2. OBJECTIVES

• Use nested lists, nested loops, totals, and averages.

3. ASSUMPTIONS

Each row represents a student and each column represents a subject. Marks are out of 100.

4. ALGORITHM

1. Create a matrix containing student marks.
2. Calculate each student's total and average.
3. Calculate each subject's average.
4. Identify the student with the highest total.
5. Display all results.

5. FLOWCHART (TEXT REPRESENTATION)

```
START
  ↓
Input or initialize marks matrix
  ↓
Calculate row totals and averages
  ↓
Calculate column averages and top student
  ↓
Display report → STOP
```

6. PSEUDOCODE

```
START
marks = student marks matrix
FOR each student
    CALCULATE total and average
FOR each subject
    CALCULATE subject average
FIND student with maximum total
DISPLAY totals, averages, and top student
STOP
```

7. PYTHON PROGRAM

```
marks = [
    [80, 75, 90],
    [65, 85, 70],
    [90, 95, 88]
]
names = ["Student 1", "Student 2", "Student 3"]

for i in range(len(marks)):
    total = sum(marks[i])
    average = total / len(marks[i])
    print(names[i], "Total:", total, "Average:", average)

for j in range(len(marks[0])):
    total = 0
    for i in range(len(marks)):
        total += marks[i][j]
    print("Subject", j + 1, "Average:",
          round(total / len(marks), 2))

totals = [sum(row) for row in marks]
top = totals.index(max(totals))
print("Top performer:", names[top])
print("Highest total:", totals[top])
```

8. TEST CASE

Use the three students' marks in the program.

9. OUTPUT

```
Student 1 Total: 245 Average: 81.66666666666667
Student 2 Total: 220 Average: 73.33333333333333
Student 3 Total: 273 Average: 91.0
Subject 1 Average: 78.33
Subject 2 Average: 85.0
Subject 3 Average: 82.67
Top performer: Student 3
Highest total: 273
```

10. CONCLUSION

The program analyses student marks using a matrix and identifies the top performer and subject averages.

CASE STUDY 49: TUPLE-BASED PRODUCT CATALOGUE AUDIT

1. PROBLEM STATEMENT

Store product details in tuples, identify invalid records, calculate inventory value, and determine which products need replenishment.

2. OBJECTIVES

• Use tuples and tuple unpacking.
• Validate product prices and stock quantities.
• Calculate total inventory value.
• Identify products with low stock.

3. ASSUMPTIONS

Each tuple contains product ID, price, and quantity. Replenishment is required when stock is below 10 units.

4. ALGORITHM

1. Store product records in a list of tuples.
2. Unpack each tuple into ID, price, and quantity.
3. Check whether the price and quantity are valid.
4. Calculate inventory value as price multiplied by quantity.
5. Identify products with stock below 10.
6. Display the audit report.

5. FLOWCHART (TEXT REPRESENTATION)

```
START
  ↓
Read product tuples
  ↓
Unpack product ID, price, and quantity
  ↓
Are price and quantity valid?
  ↓
No: flag invalid record. Yes: calculate inventory value and check stock.
  ↓
Display audit report → STOP
```

6. PSEUDOCODE

```
START
INPUT product tuples
FOR each tuple
    UNPACK ID, price, quantity
    IF price < 0 OR quantity < 0
        PRINT "Invalid record"
    ELSE
        value = price * quantity
        DISPLAY value
        IF quantity < 10
            PRINT "Replenishment required"
STOP
```

7. PYTHON PROGRAM

```
products = [
    ("P101", 100, 20),
    ("P102", 250, 5),
    ("P103", -50, 8),
    ("P104", 80, 15)
]
total_value = 0

for product in products:
    pid, price, quantity = product
    print("\nProduct ID:", pid)

    if price < 0 or quantity < 0:
        print("Invalid record")
        continue

    value = price * quantity
    total_value += value
    print("Price:", price)
    print("Quantity:", quantity)
    print("Inventory value:", value)
    if quantity < 10:
        print("Replenishment required")
    else:
        print("Stock sufficient")

print("\nTotal inventory value:", total_value)
```

8. TEST CASE

Use the four product tuples in the program.

9. OUTPUT

```
Product ID: P101
Price: 100
Quantity: 20
Inventory value: 2000
Stock sufficient

Product ID: P102
Price: 250
Quantity: 5
Inventory value: 1250
Replenishment required

Product ID: P103
Invalid record

Product ID: P104
Price: 80
Quantity: 15
Inventory value: 1200
Stock sufficient

Total inventory value: 4450
```

10. CONCLUSION

The program uses tuples to audit product records, calculate inventory value, and identify products requiring replenishment.